Sava Victor, 311CA

Documentation for PCLP3 project

This project was a collaboration between me and my classmate, Balteanu Cristian.

For the implementation of my part of the project I mainly used lab 4 and 5 but also a lot of the documentation for numpy, pandas and scikit-learn.

The dataset:

Classification is widely used for loan prediction so the dataset we chose is a loan approval prediction dataset from Kaggle, which is used to build a yes/no loan approval model. The dataset contains 4269 entries with 13 relevant columns such as `loan_id`, `no_of_dependents`, `education`, `self_employed`, `income_annum`, `loan_amount`, `loan_term`, `cibil_score`, `residential/commercial/luxury/bank_asset_value`.

Taking into account the method of dataset creation, I decided to add a column for loan to annual income ratio, delete 5% of the values of the Credit Score column, add +-5% noise for the asset value columns, and finally add a column for total asset value.

To split the dataset I used scikit-learn's built-in function `train_test_split` to split the dataset 80/20. After splitting, I saved the datasets to 2 separate csv files and sent them to my colleague.

Link to Kaggle dataset:
https://www.kaggle.com/datasets/architsharma01/loan-approval-prediction-dataset

How my project is organised:

- **`split_dataset.py:`** This file loads the dataset, simulates deletion of 5% of values of column `cibil_score`, inserts a new column for loan to annual income ratio and a new column for total asset value. After, the data is split, 2 csv files are created, then, the model is trained. At the end, a comprehensive evaluation of the model is displayed.
- **`dataset_analysis.py:`** This file contains all the exploratory data analysis graphs and tables like: Number and percentage of missing values for each column(the values are written 'missing_values_data.txt'), Descriptive statistics(the values are written in 'descriptive_statistics.txt'), Variable distribution analysis, Outlier detection, correlation matrices and finally scatter plots for analysis of relation between numerical and target columns. (Most plt.show() functions are commented, so to visualise data, make sure to uncomment where needed)
- **`app.py:`** This file contains the code for the graphical interface that can predict whether the details introduced about an applicant can estimate if the loan will be approved or not. (It is easier to use if the public URL link for the interface is accessed)
- **`Other files:`** loan_approval_dataset.csv, Readme.pdf, test.csv, train.csv

Explanatory dataset analysis:

a) **Missing values:**
   - Train: 167 entries, 4.89%
   - Test: 46 entries, 5.38%

   Solutions for missing values:
   - Completing missing values with the median or mean of the column
   - Training a model to account for missing values

b) **Descriptive statistics:**
   - Train:
   - Numerical:

- income_annum:
  - Mean: 5.061.845
  - Minimum: 200.000
  - Maximum: 9.900.000
- loan_amount:
  - Mean: 15.173.620
  - Minimum: 300.000
  - Maximum: 39.500.000
- loan_ratio:
  - Mean: 2.99
  - Minimum: 1.5
  - Maximum: 4
- cibil_score:
  - Mean: 600.97
  - Minimum: 300
  - Maximum: 900
- total_asses_value:
  - Mean: 32.569.410
  - Minimum: 388.206
  - Maximum: 91.478.590

- Categorical:

	education	self_employed	loan_status
count	3415	3415	3415
unique	2	2	2
top	Graduate	Yes	Approved
freq	1712	1733	2120

-
-
- Test:
- Numerical:
- income_annum:
  - Mean: 5.048.244
  - Minimum: 200.000
  - Maximum: 9.900.000
- loan_amount:
  - Mean: 14.972.830
  - Minimum: 400.000
  - Maximum: 38.200.000
- loan_ratio:
  - Mean: 2.96
  - Minimum: 1.67
  - Maximum: 4
- cibil_score:
  - Mean: 600.79
  - Minimum: 300
  - Maximum: 900
- total_asses_value:
  - Mean: 32.615.090

- Minimum: 799.018
- Maximum: 85.773.540

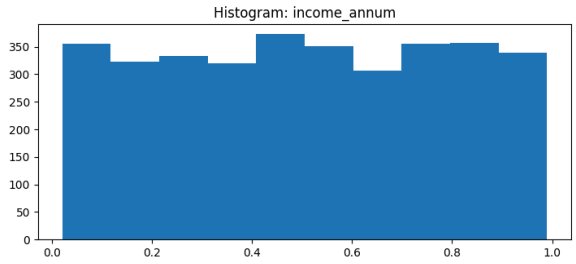
- Categorical:

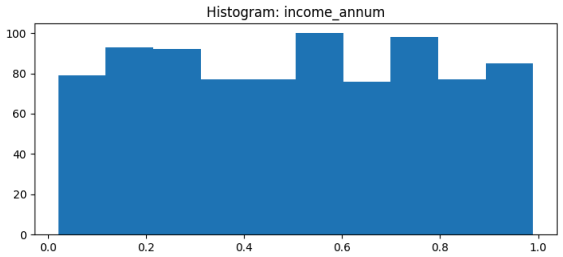
	education	self_employed	loan_status
count	854	854	854
unique	2	2	2
top	Graduate	No	Approved
freq	432	437	536

-

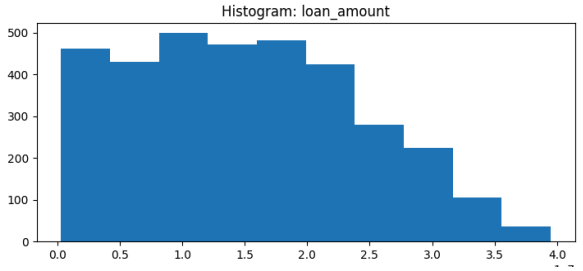
c) Variable distribution analysis

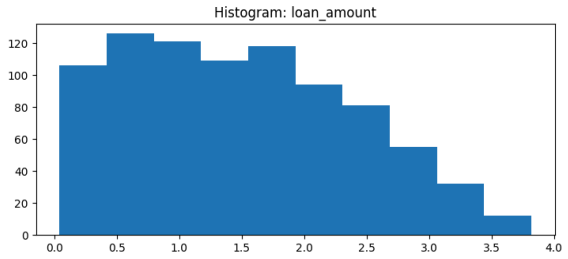
- **Train** **Test:**



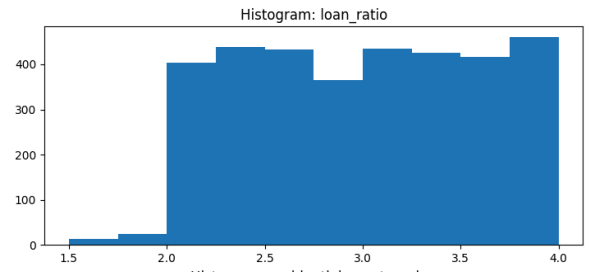


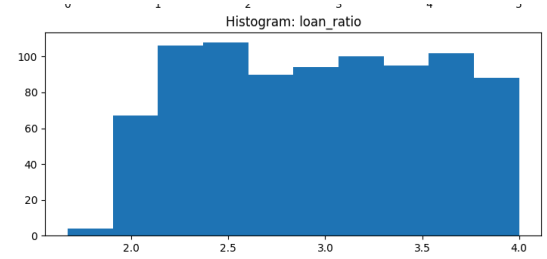
Scale: 10^7
Observation: Income distribution looks a little asymmetric to the right.
Suspicions/Ideas: There might be some very high incomes that could be considered outliers.
Preprocessing: Scaling or normalizing the values.



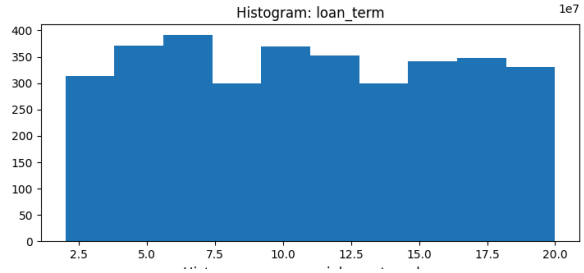


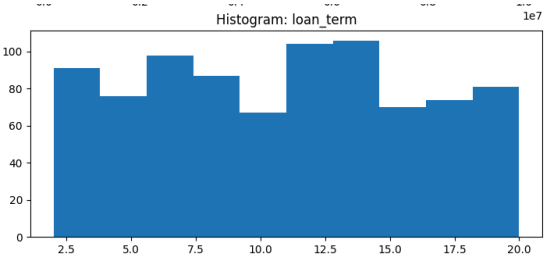
Scale: 10^7
Observation: Most loans are around 10-15 million, with a noticeable decrease for very large values.
Suspicions/Ideas: There may be a few unusually large loan applications that are impacting the distribution.
Preprocessing: Scaling or normalization.



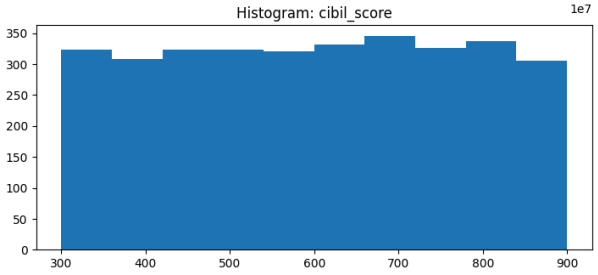


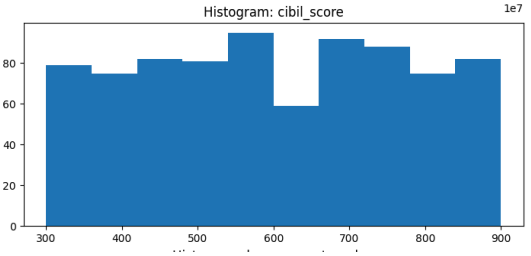
Observation: Many values are concentrated in the range of 2–3, which suggests a reasonably proportional relationship between income and debt.
Suspicions/Ideas: A very high ratio may signal customers with higher risk of not paying.
Preprocessing: Possible creation of labels based on thresholds(low, medium, high risk)





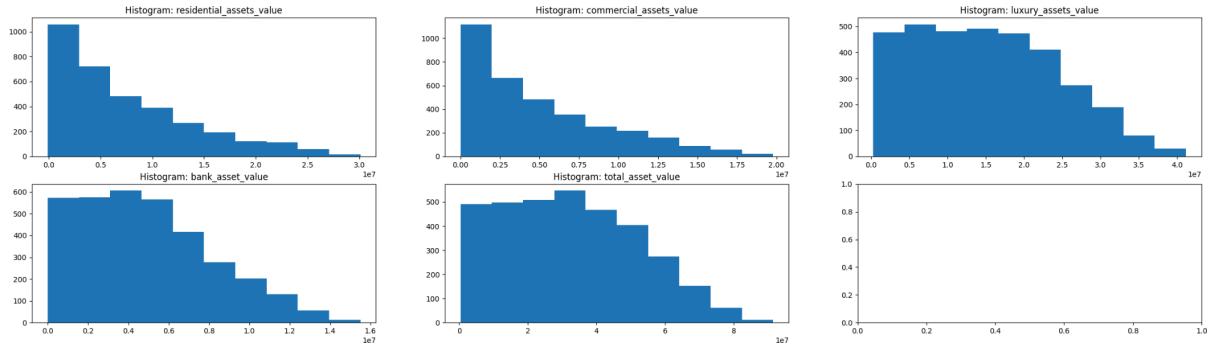
Observation: Most loans are centered around 10-15 years.
Suspicions/Ideas: There might be some very long term loans that could be considered outliers.
Preprocessing: Scaling or normalizing the values.



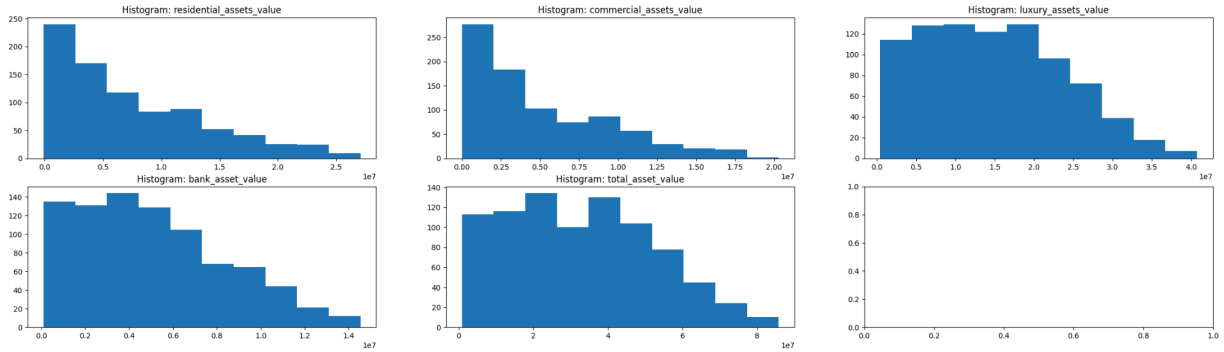


Observation: Distribution is relatively uniform, but some data is missing.
Suspicions/Ideas: Customers without a score may be rejected more often.
Preprocessing: Imputation (mean/median) or training separate model for missing data.

Train:

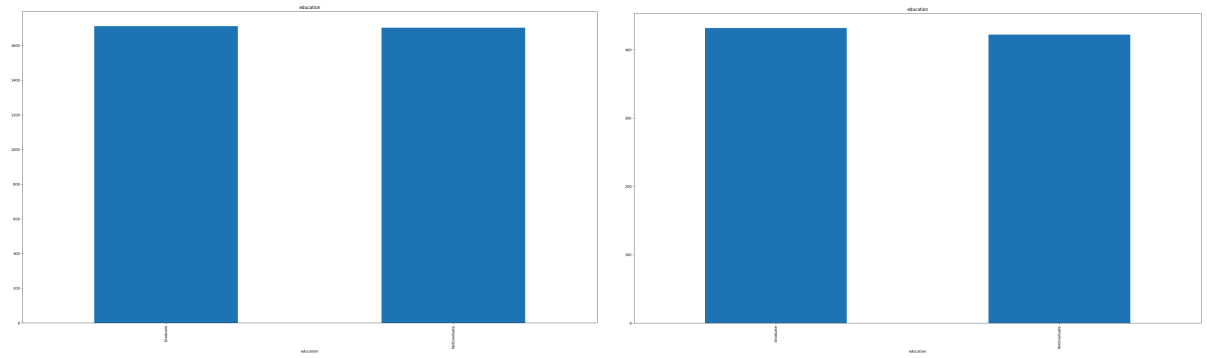


Test:

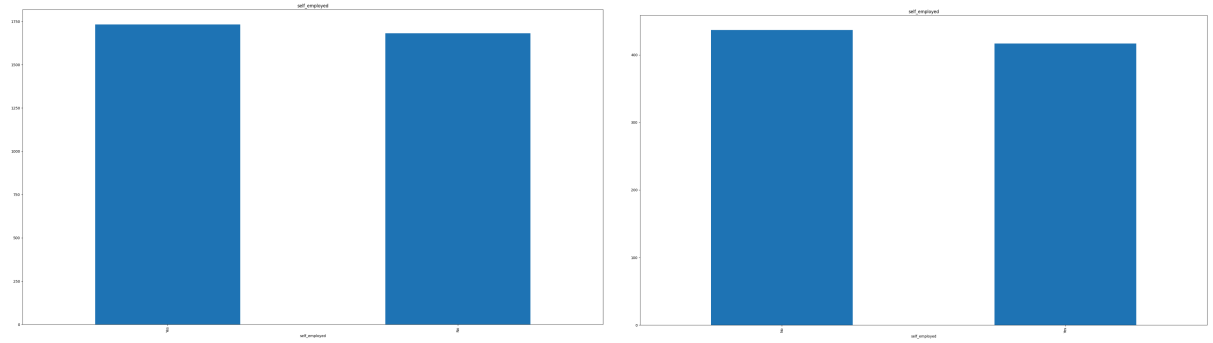


Scale: 10^7
Observation: Values concentrated in a range, but with long tails to the right.
Suspicions/Ideas: Possible clients with exaggerated net worth who could influence the models.
Preprocessing: Normalization or elimination of the extreme 1-2%.

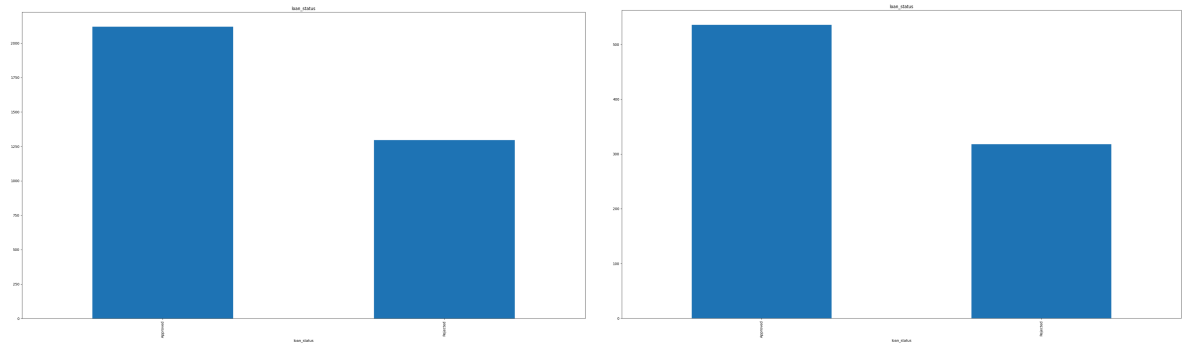
Education countplot:



Self-employed countplot:



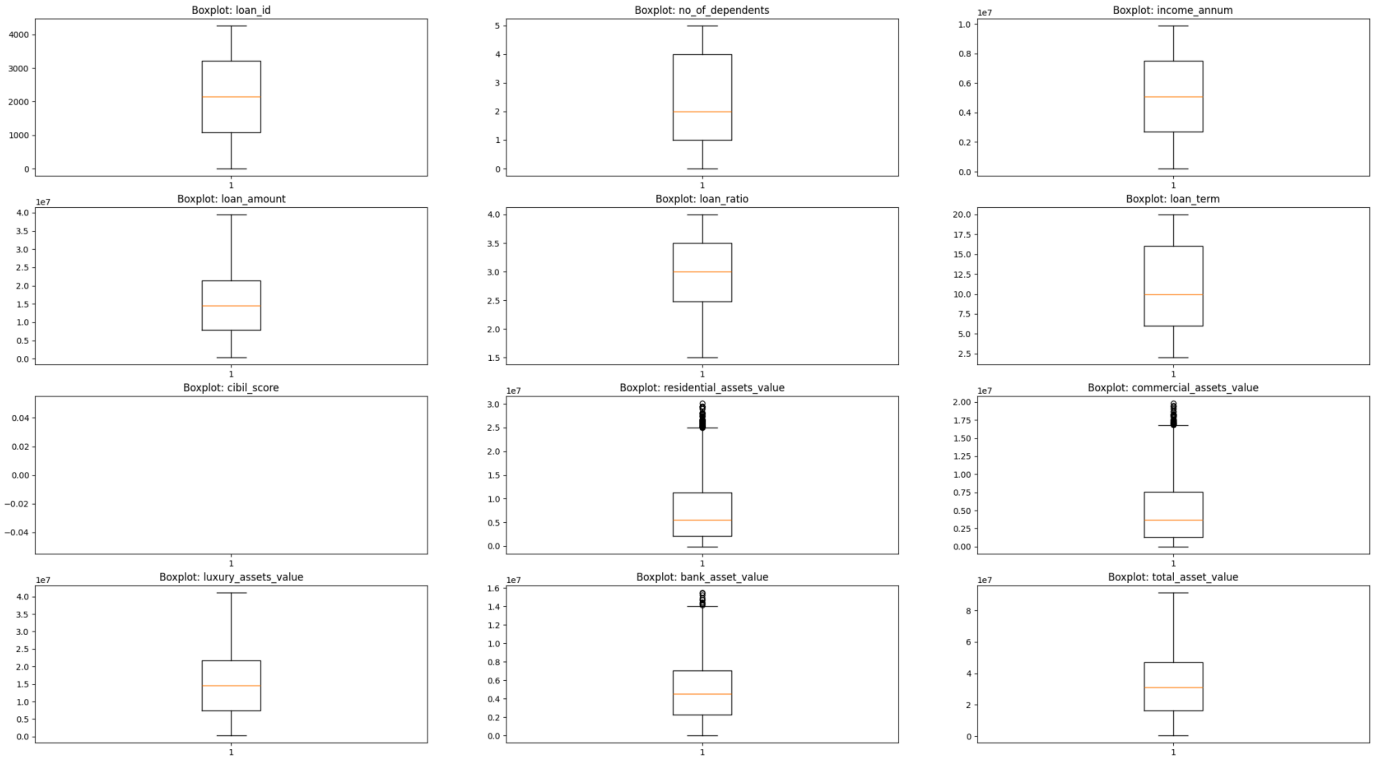
Loan Status countplot:



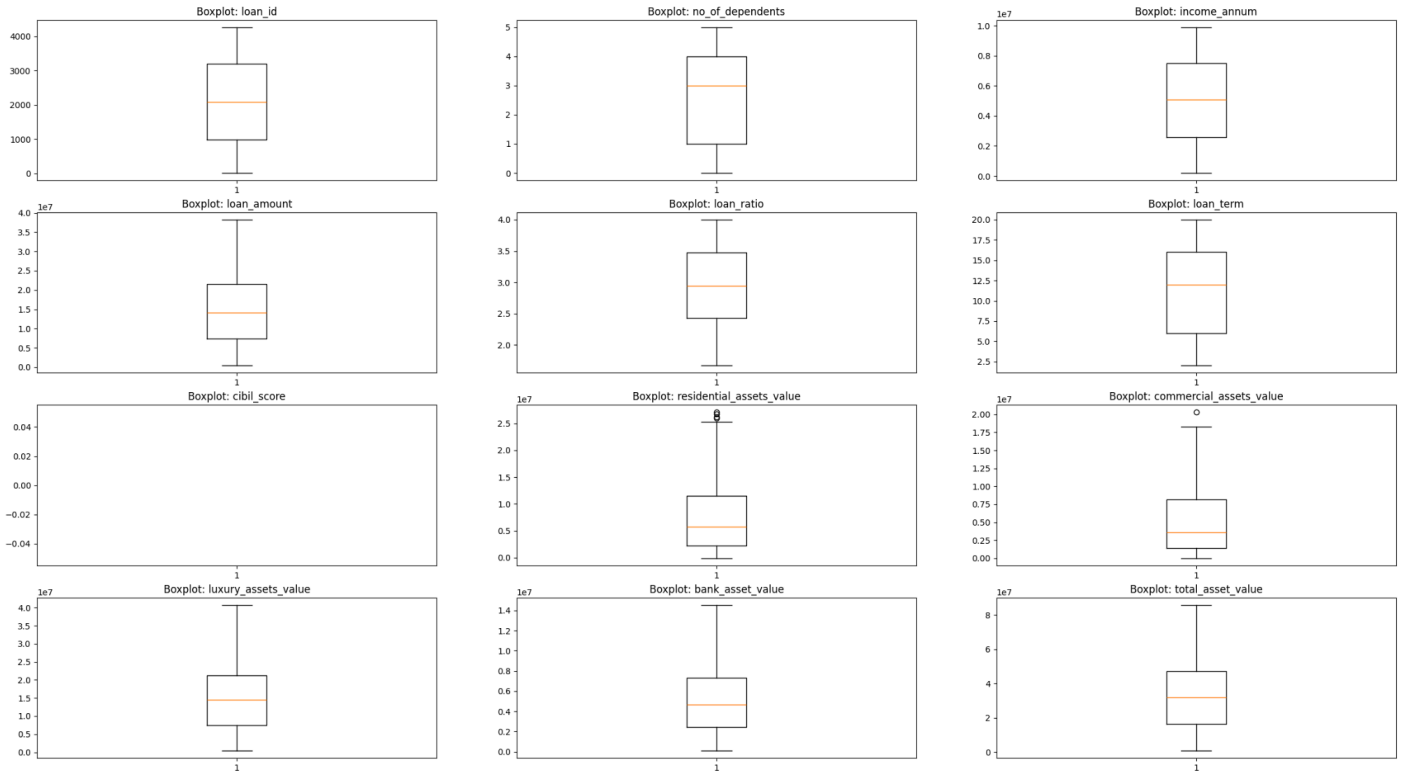
Observation for countplots: Although the distribution in the education and self-employment columns is very balanced, most of the requests for the loans are approved.
Suspicions/Ideas: There is a class imbalance that can influence the training of classification models.
Preprocessing: Balancing techniques (SMOTE, undersampling)

d) Outlier values

Train:



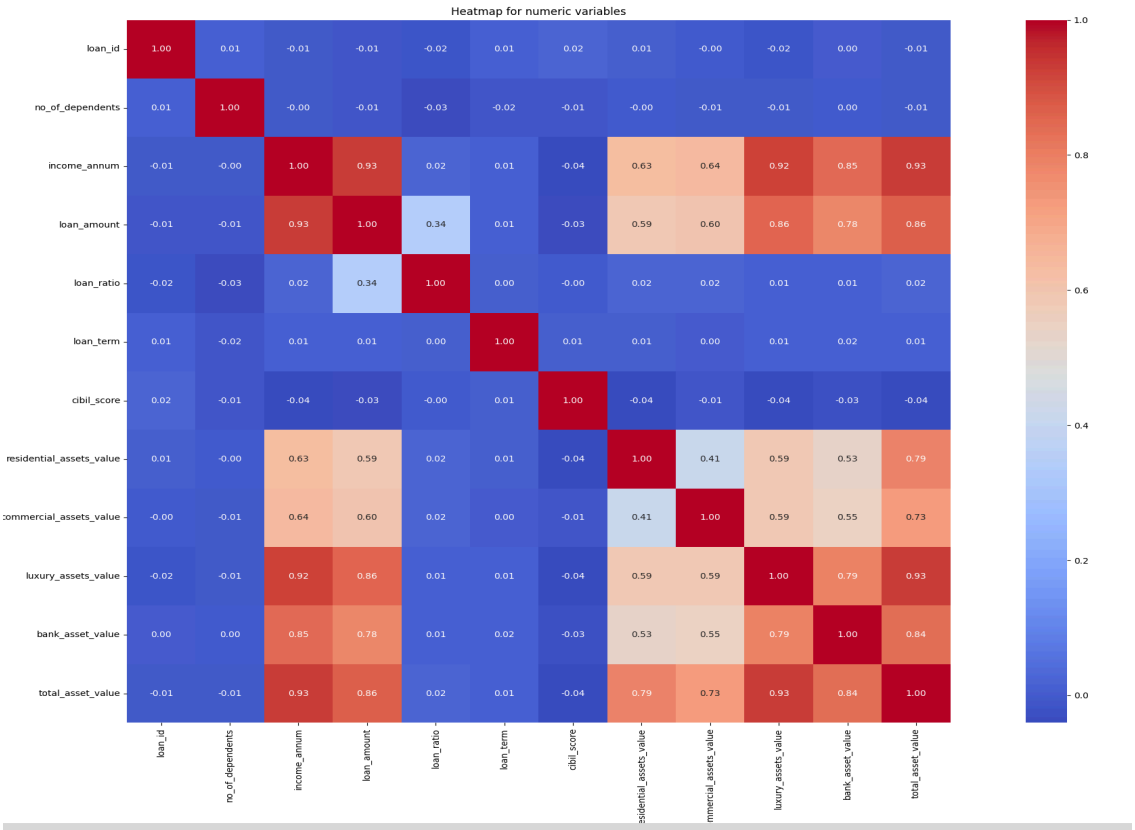
Test:



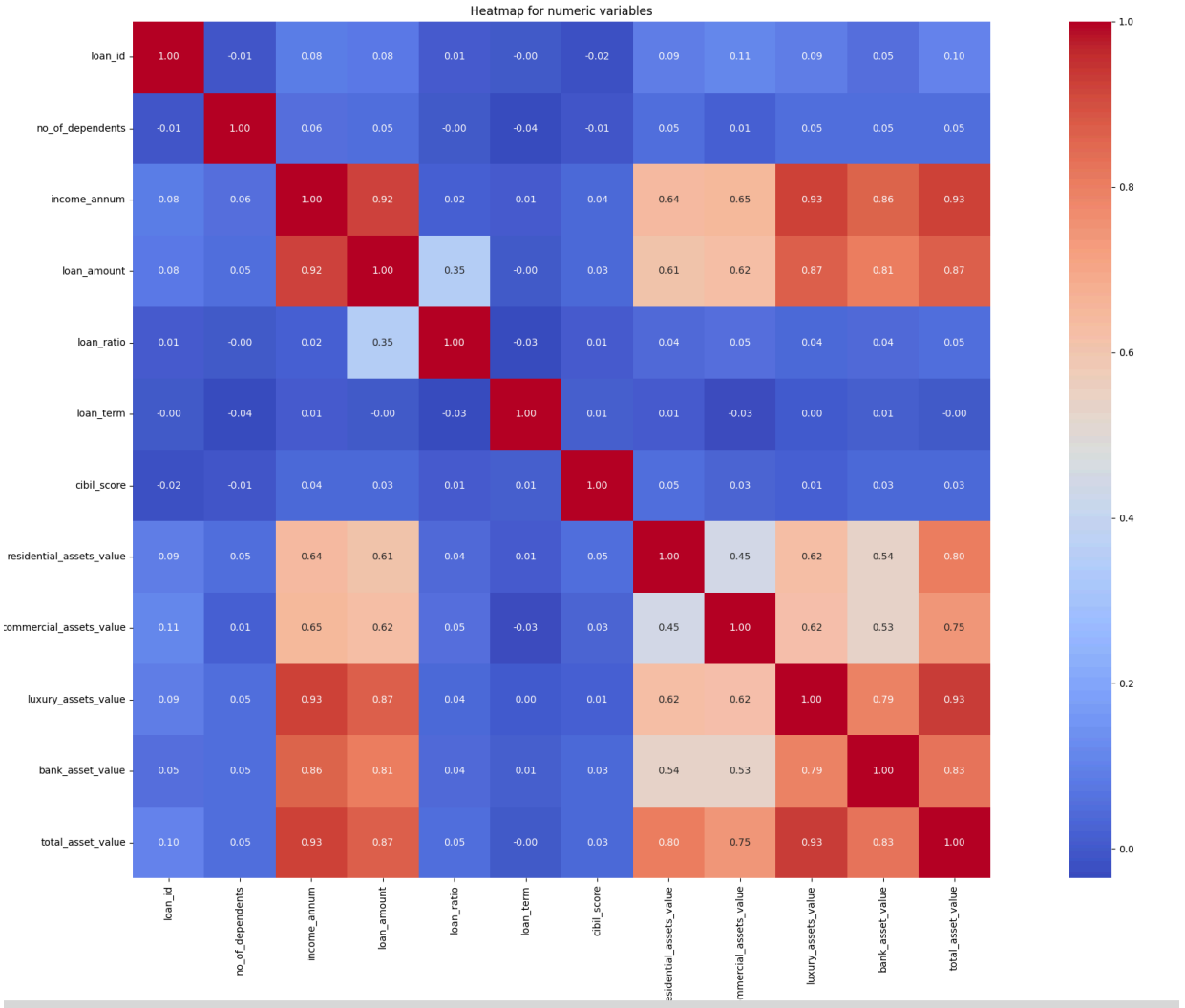
Observation for boxplots: Most outlier values are found at `residential/commercial/bank_asset_value` columns. So, columns with artificially added values (noise) generate visible outliers. This can also be seen on the test dataset, but on a smaller scale.
Suspicions/Ideas: These outliers can influence the performance of the training model.
Preprocessing: Normalization

e) Heatmap(Confusion matrix)

Train:



Test:



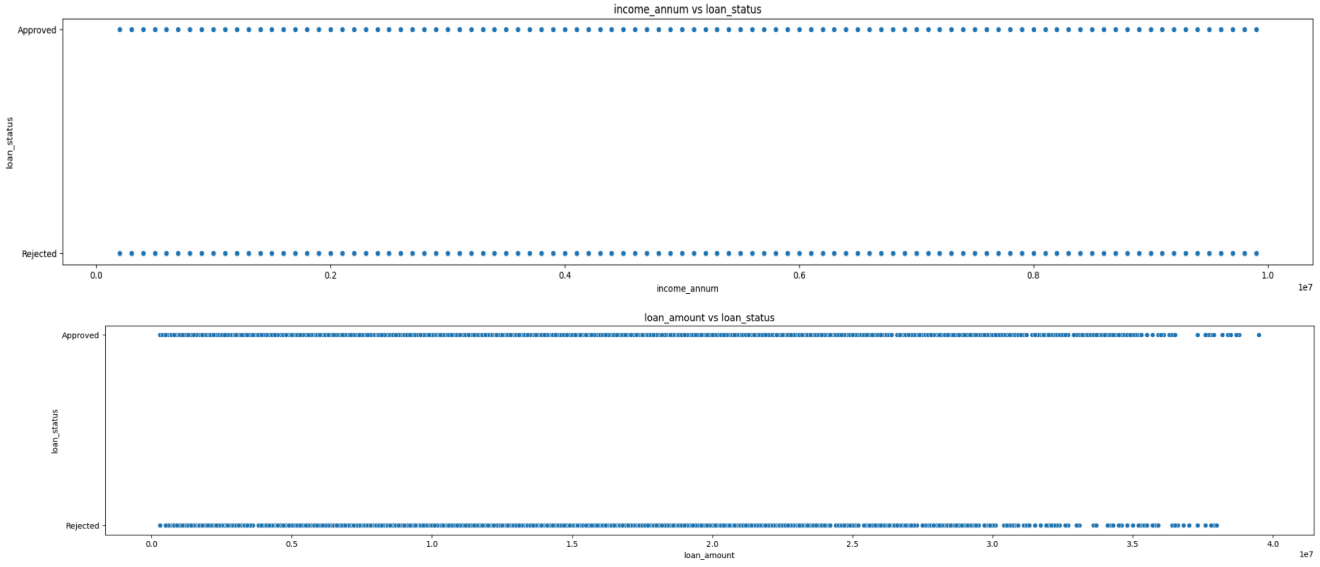
Observation: The variables `loan_amount` and `income_annum` are highly correlated. `total_asset_value` has high correlations with its components.

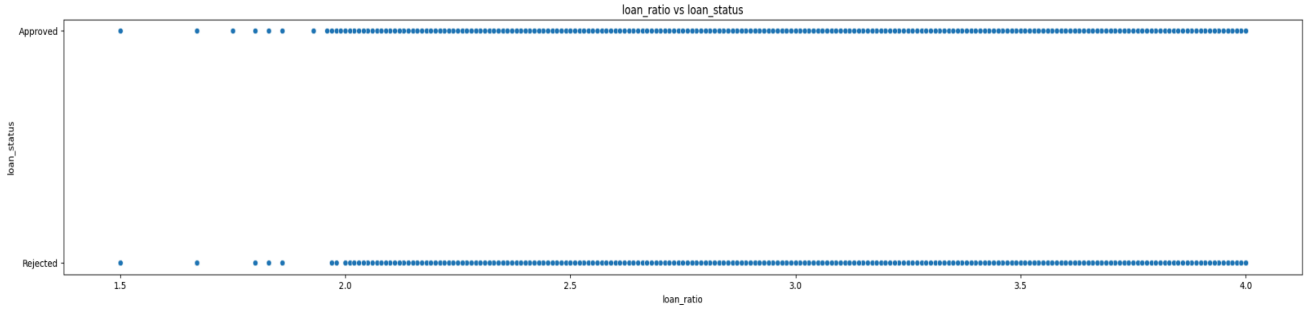
Suspicions/Ideas: Redundancy between some variables (total_asset_value ~= sum of 4 others).

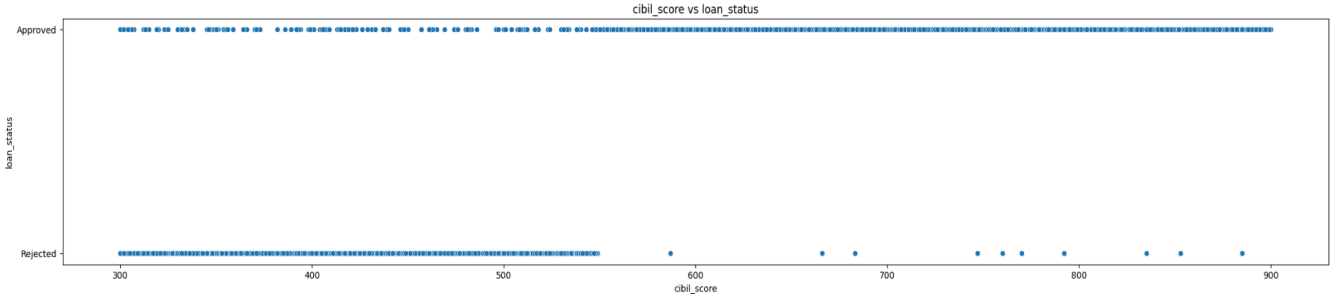
Preprocessing: Normalization or standardisation of asset columns.

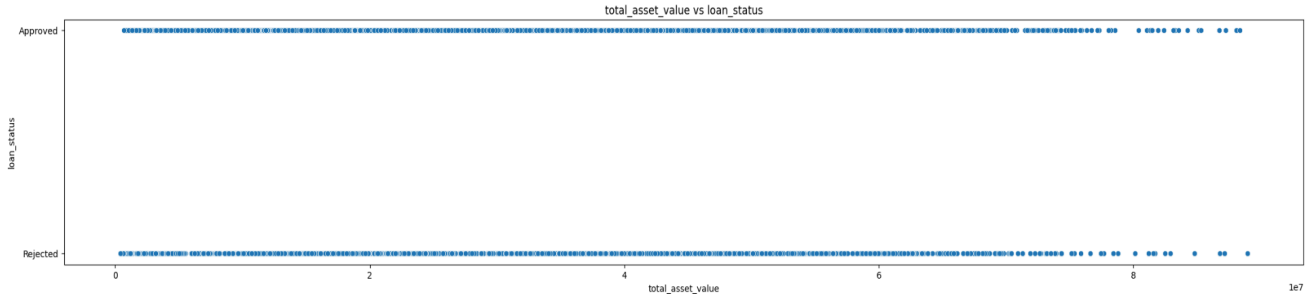
f) Scatter plots

Train:

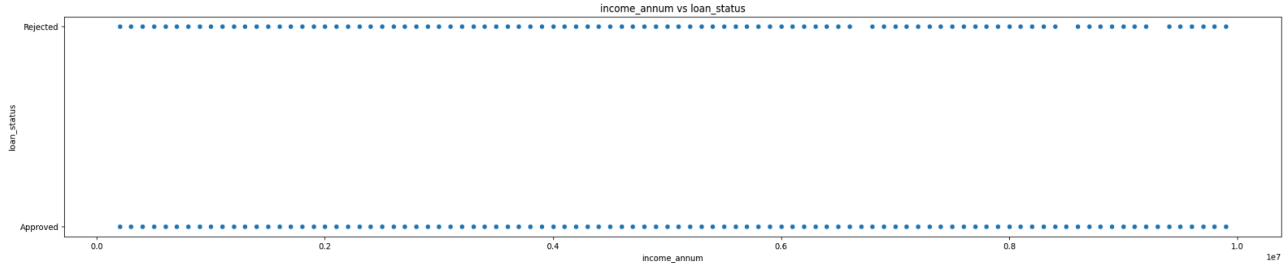


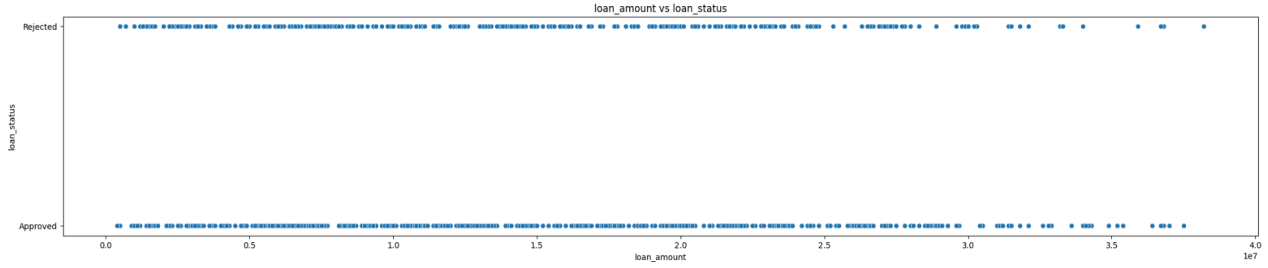

loan_ratio vs loan_status
Approved
Rejected
loan_status
1.5
2.0
2.5
3.0
3.5
4.0
loan_ratio

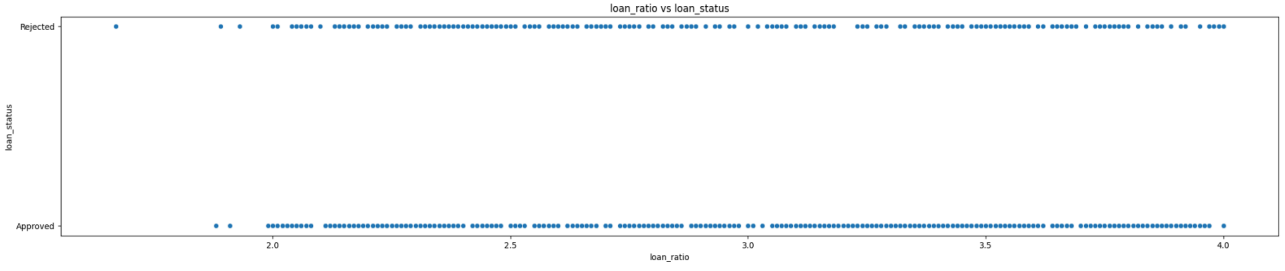

cibil_score vs loan_status
Approved
Rejected
loan_status
300
400
500
600
700
800
900
cibil_score

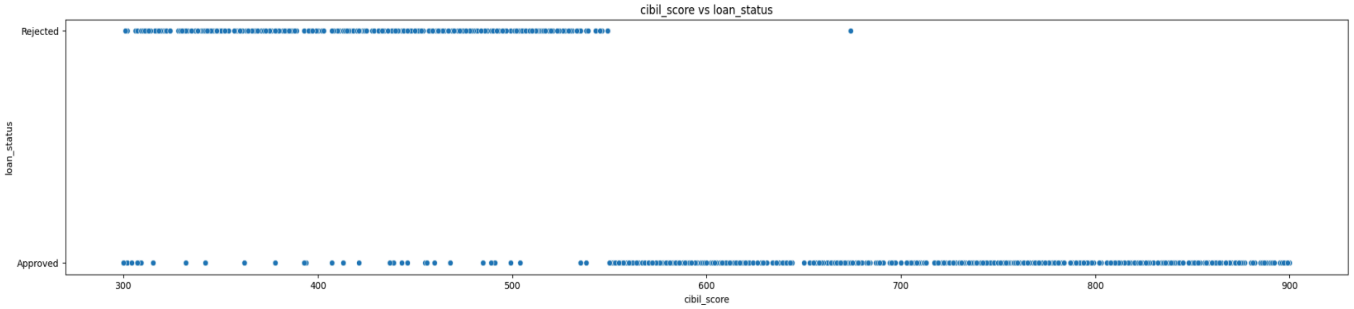

total_asset_value vs loan_status
Approved
Rejected
loan_status
0
2
4
6
8
1e7
total_asset_value

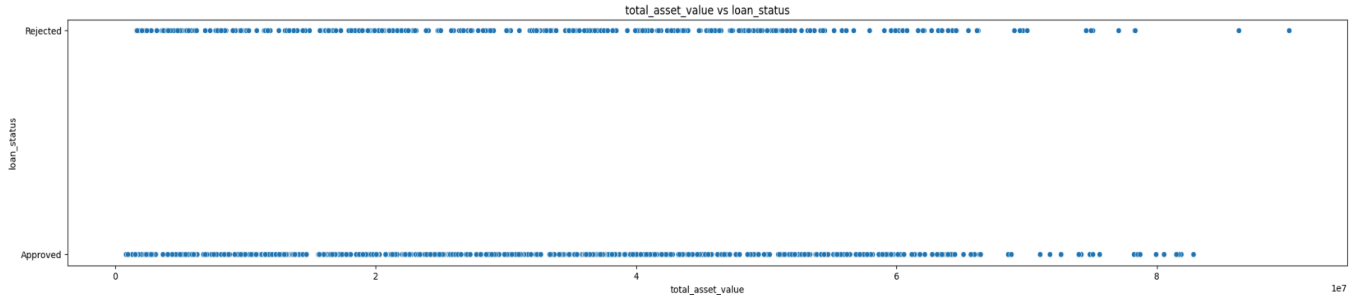
Test:


income_annum vs loan_status
Rejected
Approved
loan_status
0.0
0.2
0.4
0.6
0.8
1.0
1e7
income_annum


loan_amount vs loan_status
Rejected
Approved
loan_status
0.0
0.5
1.0
1.5
2.0
2.5
3.0
3.5
4.0
1e7
loan_amount


loan_ratio vs loan_status
Rejected
Approved
loan_status
2.0
2.5
3.0
3.5
4.0
loan_ratio





Observation:
If the points are concentrated towards 1 for large values -> possible positive predictor.
If it's a random scatter -> weak relationship.
Strong relations: For the `loan_amount` variable, we see that larger loans tend to be approved less often (concentration in the 0 zone). For `cibil_score`, a clear trend is observed: higher scores correlate with approvals (`loan_status`=1).
Weak relations: No discernible correlation for `income_annum. Loan_ratio` feature does not seem to have a clear influence. The same for `total_asset_value`
Suspicions/Ideas:
For example, for `loan_amount`, approvals may be concentrated at average values, not high or low ones -> suggesting a nonlinear relationship.
If for `cibil_score` we notice that most approvals appear only above 600, we can suspect that the model would learn well based on this threshold.
Combined predictors would have more predictive power. For example: `loan_ratio` combines two important variables, but can be improved with `loan_term` or `cibil_score`.
Preprocessing: Standard recommendations: Normalization / Standardization. How?
Apply StandardScaler or MinMaxScaler for `income_annum`, `loan_amount` and `total_asset_value` because they are on very different scales.
Creating labels:
`cibil_score` >= 650 -> good_credit = 1 - can help simple models capture relationships more easily.

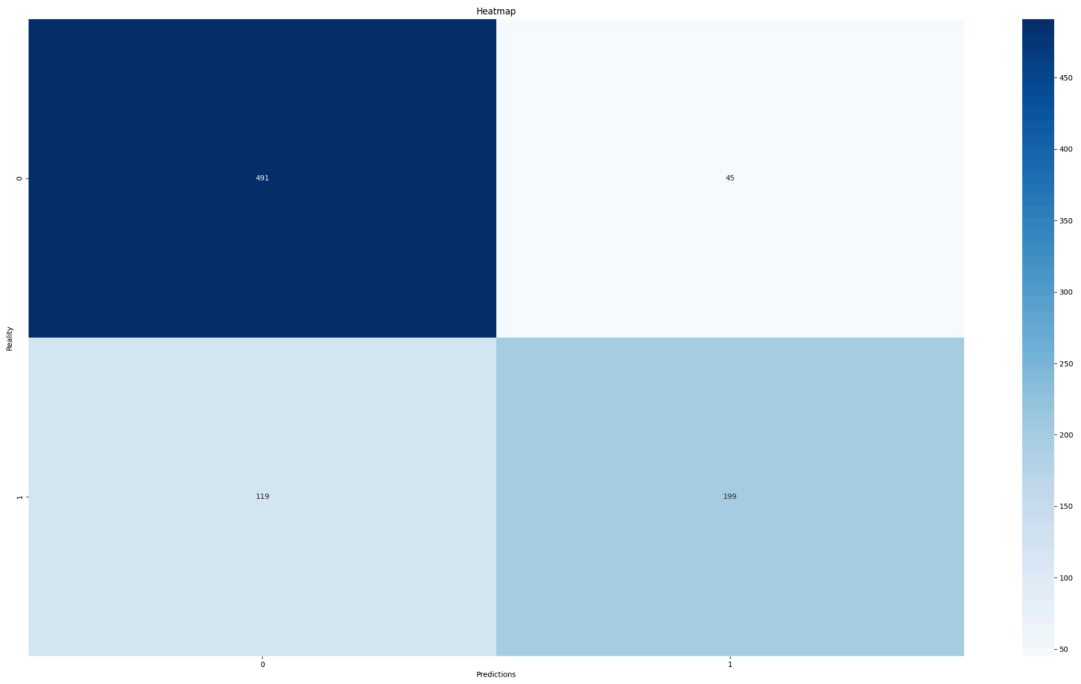
Training model:

- **Classification model:** Logistic Regression (maximum 500 iterations)

- **Model performance:**

	precision	recall	f1-score
False	0.81	0.91	0.86
True	0.81	0.63	0.71
accuracy			0.81

- **Precision**
  - **The proportion of positive predictions that are correct.**
  - **True:** 0.81 -> of all the loans that the model predicted to be approved, 81% were actually approved.
  - **False:** 0.81 -> of all the rejection predictions, 81% were correct.
- **Recall**
  - **The proportion of real instances that were correctly predicted.**
  - **True:** 0.63 -> only 63% of real approvals were correctly identified by the model.
  - **False:** 0.91 -> the model identified rejections very well (91%).
- **F1-score**
  - **Harmonic average of precision and recall - balance between them.**
  - **True:** 0.71 -> moderate performance on the approval class.
  - **False:** 0.86 -> good performance on the rejection class.
- **Accuracy**
  - **The total percentage of correct predictions.**
  - **0.81** (81%) -> the model makes correct predictions 81% of the time.
  - While the accuracy is good (81%), the model is conservative and tends to reject rather than approve. This can be useful in the context of cautious banks, but is risky if the goal is to provide more eligible loans.
- **Relevant graphs:**
- **Heatmap:**



Observation: The model predicts class 0 (rejected loans) well: 491 out of 536 (~92%). On class 1 (approved loans), it struggles: only 199 correct, 119 wrong. The model seems to favor negative predictions (bias towards 0).

Suspicions/Ideas: The dataset is slightly unbalanced (more approvals than rejections). The model performs worse on the positive class (1). It may not have enough variables with predictive power for approvals.
Preprocessing: Normalization or standardisation of income, loan amount and asset columns.

- **What could be improved?**
- In the future, the model could be improved by applying techniques such as standardizing numerical values, removing outliers from asset values, logarithmic transformations for asymmetric variables, and balancing classes during training. It would also be useful to test more powerful models such as Random Forest, which can better capture interactions between variables.

Final notes:

This project was a good opportunity to understand the practical steps of a complete machine learning workflow - from cleaning and expanding a real dataset, to training and evaluating a classification model. Through detailed exploratory analysis, I gained a clear picture of the data structure and possible relationships between variables. Although the resulting model provided decent results, there is room for future improvements through more advanced preprocessing, class balancing, and testing more powerful algorithms. I believe that this project significantly contributed to deepening my knowledge of data processing and modeling in Python, and laid the foundation for more complex applications in the future.